Name	Divyansh Dwivedi
Roll number	S24CSEU0854
Batch	B43
Date of laboratory session	22 September 2026
Practical	P05 — Settings in a File, Bugs Caught by a Robot

Practical 05 --- Settings in a File, Bugs Caught by a Robot

YAML configuration and automated testing with pytest

SCSE3040 Machine Learning Operations · Bennett University · Session 2026-27

Lectures this follows	L08
Course Outcome	CO3
Lab duration	120 minutes
Memory needed	about 350 MB (fine on a 4 GB or 8 GB machine)
Extra software	nothing beyond the course venv
Marks	10

Aim

1. Move every hardcoded setting out of your code and into a YAML file.
2. Read that file from Python and use it to control training.
3. Write automated tests that check your code without you watching.
4. Run the whole test suite with one command and read its report.

What you need to know first

Two habits today. They look unrelated. They are both about the same thing: being able to change your project safely.

Configuration. Your code is full of numbers somebody chose: the test size of 0.2, the seed 42, the four column names. Right now they are typed into the middle of your code. That is called a **magic number**, and it causes two problems. To change one you must edit code, which risks breaking it. And to find out what settings produced a result, someone must read every line.

The fix is to put settings in a separate file. We use **YAML** (YAML Ain't Markup Language), a format designed to be read by humans. Indentation makes the structure; no brackets, no commas. It is what Docker, Kubernetes and GitHub Actions all use, so you will meet it constantly from P08 onwards.

Testing. Right now you check your code by looking at output and thinking "that seems about right". That works until your project has forty functions, and then it stops working entirely.

A **test** is a small piece of code that checks another piece of code, and says PASS or FAIL without a human. **pytest** is the tool that finds all your tests and runs them. One command, and you know whether anything broke.

The magic word is `assert`. `assert x == 5` does nothing at all if `x` is 5, and raises an error if it is not. A test is just a function whose name starts with `test_`, containing asserts.

Why this matters for MLOps: in P11 a server will run these tests automatically every time you change your code, and refuse to deploy if any of them fail.

Before you start

- Practical P04 is finished. You can write a module and import it.
- You know what a function is and what it means for one to return a value.

How to run a cell

Click on a grey code cell, then press **Shift + Enter**. The cell runs and the cursor moves to the next one. A number appears in the `[ ]` on the left when the cell has finished.

Run the cells in order, from the top. A later cell almost always uses something an earlier cell created. If you jump ahead you will see a `NameError`, which just means "you have not made that thing yet".

If everything goes wrong, use the menu: **Kernel -> Restart Kernel and Clear All Outputs**, then start again from the first cell. Nothing is damaged by doing this.

In [1]:

```
# Step 0 --- check the workbench before we start.
# This cell only looks; it changes nothing. Run it and read the last line.

import sys
from pathlib import Path

print("Python  :", sys.version.split()[0])
print("Folder  :", Path.cwd().name)

_missing = []
for _name in ['numpy', 'pandas', 'sklearn', 'yaml', 'pytest']:
    try:
        __import__(_name)
    except ImportError:
        _missing.append(_name)

for _name in ['numpy', 'pandas', 'sklearn', 'yaml', 'pytest']:
    _mark = "missing" if _name in _missing else "ok"
    print(f"  {_name:<14} {_mark}")

if _missing:
    print()
    print("STOP. Some libraries are missing:", ", ".join(_missing))
    print("Ask your instructor to run the setup in labs/SETUP.md.")
else:
    print()
    print("All good. You can carry on to Step 1.")
```

```
Python  : 3.13.15
Folder  : content
  numpy          ok
  pandas         ok
  sklearn        ok
  yaml           ok
  pytest         ok

All good. You can carry on to Step 1.
```

Step 0b --- the dataset

Every practical in this course uses the same 600 food deliveries. The next cell makes sure the file is there.

In [2]:

```
# The delivery-time dataset every practical in this course uses.
# If the file is missing we build it again from the same seed, so every
# student in the room gets byte-for-byte the same 600 rows.

import csv
from pathlib import Path

import numpy as np

SEED = 42
N_ROWS = 600
DATA = Path("..") / "data" / "delivery_times.csv"


def make_delivery_csv(path=DATA):
    """Write the 600-row delivery dataset. Same formula as the lectures."""
    rng = np.random.default_rng(SEED)
    distance_km = np.round(rng.uniform(0.5, 12.0, N_ROWS), 2)
    prep_time_min = np.round(rng.uniform(5, 30, N_ROWS), 0)
    traffic_level = rng.integers(1, 4, N_ROWS)
    rain = rng.binomial(1, 0.25, N_ROWS)
    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["distance_km", "prep_time_min", "traffic_level",
                    "rain", "delivery_min"])
        for i in range(N_ROWS):
            w.writerow([distance_km[i], int(prep_time_min[i]),
                        int(traffic_level[i]), int(rain[i]), delivery_min[i]])
    return path


if not DATA.exists():
    make_delivery_csv()
    print("dataset rebuilt ->", DATA)
else:
    print("dataset found   ->", DATA)
```

```
dataset rebuilt -> ../data/delivery_times.csv
```

Walkthrough

Read each step, then run its cell.

Step 1 --- Look at the problem: settings buried in code

Here is training code in the style you have written so far. Count the numbers and names that somebody chose. There are seven.

In [3]:

```
from pathlib import Path

import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split

WORK = Path("work")
WORK.mkdir(exist_ok=True)

orders = pd.read_csv(DATA)
X = orders[["distance_km", "prep_time_min", "traffic_level", "rain"]]
y = orders["delivery_min"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
                                          random_state=42)
mae = mean_absolute_error(y_te, LinearRegression().fit(X_tr, y_tr)
                          .predict(X_te))
print(f"MAE {mae:.2f}")
```

```
MAE 1.92
```

To try a 70/30 split you must edit that line. To find out what split produced yesterday's number, you must read the code as it was yesterday. Both are avoidable.

Step 2 --- Write the settings into a YAML file

YAML rules you need today, and no more:

- `key: value` gives one setting
- indentation with **spaces** groups settings together
- `- item` makes a list
- `#` starts a comment

Never use tab characters in YAML. It is the single most common mistake, and the error message will not tell you.

In [4]:

```
config_yaml = """# Settings for the delivery-time model.
# Change values here, never in the code.

data:
  path: ../data/delivery_times.csv
  features:
    - distance_km
    - prep_time_min
    - traffic_level
    - rain
  target: delivery_min

split:
  test_size: 0.2
  seed: 42

model:
  name: linear_regression
"""

(WORK / "config.yaml").write_text(config_yaml, encoding="utf-8")
print((WORK / "config.yaml").read_text(encoding="utf-8"))
```

```
# Settings for the delivery-time model.
# Change values here, never in the code.

data:
  path: ../data/delivery_times.csv
  features:
    - distance_km
    - prep_time_min
    - traffic_level
    - rain
  target: delivery_min

split:
  test_size: 0.2
  seed: 42

model:
  name: linear_regression
```

Step 3 --- Read the file into Python

`yaml.safe_load` turns that text into ordinary Python dictionaries and lists. Always `safe_load`, never `load` --- the unsafe version can execute code hidden inside a file somebody sent you.

In [5]:

```
import yaml

with open(WORK / "config.yaml", encoding="utf-8") as fh:
    config = yaml.safe_load(fh)

print(type(config))
print()
print("features :", config["data"]["features"])
print("test size:", config["split"]["test_size"])
print("seed     :", config["split"]["seed"])
```

```
<class 'dict'>

features : ['distance_km', 'prep_time_min', 'traffic_level', 'rain']
test size: 0.2
seed     : 42
```

It is a dictionary of dictionaries. `config["split"]["seed"]` reads exactly like the file looks, which is the whole point of YAML.

Step 4 --- Train using only the config

Same training as Step 1, but now there is not one chosen number in the code. Every setting comes from the file.

In [6]:

```
def train_from_config(config, data_path):
    """Train and score, taking every setting from the config."""
    orders = pd.read_csv(data_path)
    X = orders[config["data"]["features"]]
    y = orders[config["data"]["target"]]

    X_tr, X_te, y_tr, y_te = train_test_split(
        X, y,
        test_size=config["split"]["test_size"],
        random_state=config["split"]["seed"],
    )
    model = LinearRegression().fit(X_tr, y_tr)
    return model, mean_absolute_error(y_te, model.predict(X_te))


model, mae = train_from_config(config, DATA)
print(f"MAE {mae:.2f} minutes")
```

```
MAE 1.92 minutes
```

Now change `test_size` to `0.3` in your head. You would edit one line of a settings file --- no code review, no risk of a typo breaking training.

Step 5 --- Prove it: change a setting, get a different run

Watch the config actually control the behaviour.

In [7]:

```
for test_size in [0.2, 0.3, 0.4]:
    config["split"]["test_size"] = test_size
    _, mae = train_from_config(config, DATA)
    print(f"test_size {test_size} -> MAE {mae:.2f}")

config["split"]["test_size"] = 0.2   # put it back
```

```
test_size 0.2 -> MAE 1.92
test_size 0.3 -> MAE 2.01
test_size 0.4 -> MAE 2.04
```

Step 6 --- Now testing. What is an assert?

An `assert` says "this must be true". If it is, nothing happens. If it is not, Python stops and complains.

Run this cell. The first two lines do nothing visible. The third one is caught and printed on purpose.

In [8]:

```
assert 2 + 2 == 4
assert "delivery".startswith("del")

try:
    assert 2 + 2 == 5, "arithmetic is broken"
except AssertionError as problem:
    print("AssertionError:", problem)
```

```
AssertionError: arithmetic is broken
```

That is the whole idea. A test is a function full of asserts.

Step 7 --- Write a module worth testing

Tests need something to test. Here is a small module with two functions and one deliberate rule: a delivery cannot cover zero distance.

In [9]:

```
orders_py = '''"Small helpers for delivery orders."

TRAFFIC_LEVELS = (1, 2, 3)


def minutes_per_km(delivery_min, distance_km):
    "How many minutes each kilometre took."
    if distance_km <= 0:
        raise ValueError("distance_km must be positive")
    return delivery_min / distance_km


def is_valid_order(order):
    "True if this order is usable for training."
    if order.get("distance_km", 0) <= 0:
        return False
    if order.get("prep_time_min", 0) < 0:
        return False
    if order.get("traffic_level") not in TRAFFIC_LEVELS:
        return False
    if order.get("rain") not in (0, 1):
        return False
    return True
'''

(WORK / "orders.py").write_text(orders_py, encoding="utf-8")
print("wrote", WORK / "orders.py")
```

```
wrote work/orders.py
```

Step 8 --- Write the tests

Three rules pytest follows, and they are all naming rules:

- the file must be called `test_something.py`
- each test must be a function called `test_something`
- inside, use plain `assert`

Notice the third test: it checks that a **bad** input raises an error. Testing that things fail properly is as important as testing that they work.

In [10]:

```
test_py = '''"Tests for orders.py."

import pytest

from orders import is_valid_order, minutes_per_km


def test_minutes_per_km_simple():
    assert minutes_per_km(30, 10) == 3.0


def test_minutes_per_km_rejects_zero_distance():
    with pytest.raises(ValueError):
        minutes_per_km(30, 0)


def test_good_order_is_accepted():
    good = {"distance_km": 5.0, "prep_time_min": 20,
            "traffic_level": 2, "rain": 0}
    assert is_valid_order(good) is True


def test_bad_traffic_level_is_rejected():
    bad = {"distance_km": 5.0, "prep_time_min": 20,
           "traffic_level": 9, "rain": 0}
    assert is_valid_order(bad) is False
'''

(WORK / "test_orders.py").write_text(test_py, encoding="utf-8")
print("wrote", WORK / "test_orders.py")
```

```
wrote work/test_orders.py
```

Step 9 --- Run the tests with one command

`pytest` finds every `test_*.py` file, runs every `test_*` function, and reports. A dot means a pass, an `F` means a failure.

In [11]:

```
import subprocess
import sys

def run_pytest(*extra):
    """Run pytest inside work/ and print its report."""
    done = subprocess.run(
        [sys.executable, "-m", "pytest", "-v", *extra],
        cwd=WORK, capture_output=True, text=True,
    )
    print(done.stdout[-2500:])
    print("exit code:", done.returncode,
          "(0 means every test passed)")
    return done


run = run_pytest()
```

```
============================= test session starts ==============================
platform linux -- Python 3.13.15, pytest-8.4.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /content/work
plugins: anyio-4.14.2, typeguard-4.6.0, langsmith-0.12.1
collecting ... collected 4 items

test_orders.py::test_minutes_per_km_simple PASSED                        [ 25%]
test_orders.py::test_minutes_per_km_rejects_zero_distance PASSED         [ 50%]
test_orders.py::test_good_order_is_accepted PASSED                       [ 75%]
test_orders.py::test_bad_traffic_level_is_rejected PASSED                [100%]

============================== 4 passed in 0.01s ===============================

exit code: 0 (0 means every test passed)
```

Four tests, four passes, exit code 0. That exit code is what a pipeline reads in P11 to decide whether your change is allowed to go live.

Step 10 --- Break something on purpose

A test suite you have never seen fail is a test suite you should not trust. Let us break the module and watch pytest catch it.

In [12]:

```
broken = (WORK / "orders.py").read_text(encoding="utf-8")
(WORK / "orders.py").write_text(
    broken.replace("return delivery_min / distance_km",
                   "return delivery_min * distance_km"),
    encoding="utf-8",
)

run_pytest("-x", "-q")

(WORK / "orders.py").write_text(broken, encoding="utf-8")  # repair
print()
print("module repaired")
```

```
============================= test session starts ==============================
platform linux -- Python 3.13.15, pytest-8.4.2, pluggy-1.6.0
rootdir: /content/work
plugins: anyio-4.14.2, typeguard-4.6.0, langsmith-0.12.1
collected 4 items

test_orders.py F

=================================== FAILURES ===================================
_____________________________ test_minutes_per_km_simple ______________________________

    def test_minutes_per_km_simple():
>       assert minutes_per_km(30, 10) == 3.0
E       assert 300 == 3.0
E        +  where 300 = minutes_per_km(30, 10)

test_orders.py:9: AssertionError
=========================== short test summary info ============================
FAILED test_orders.py::test_minutes_per_km_simple - assert 300 == 3.0
!!!!!!!!!!!!!!!!!!!!!!!!! stopping after 1 failures !!!!!!!!!!!!!!!!!!!!!!!!!
============================== 1 failed in 0.04s ===============================

exit code: 1 (0 means every test passed)

module repaired
```

Read the failure report. pytest shows you the line, the values it compared, and what it expected. That report is why you write tests instead of print statements.

Step 11 --- Test the model's behaviour, not its exact score

Testing machine learning has one trap: never assert `mae == 2.03`. That test breaks the day someone adds data, and it tells you nothing.

Test **behaviour** that must always hold instead. Rain must never make a delivery faster. That is true of any correct model.

In [13]:

```
model_test_py = '''"Behaviour tests for the trained model."

import pandas as pd


def make_order(distance_km=5.0, prep_time_min=20,
               traffic_level=2, rain=0):
    return pd.DataFrame([{
        "distance_km": distance_km,
        "prep_time_min": prep_time_min,
        "traffic_level": traffic_level,
        "rain": rain,
    }])


def test_rain_never_makes_delivery_faster(trained_model):
    dry = trained_model.predict(make_order(rain=0))[0]
    wet = trained_model.predict(make_order(rain=1))[0]
    assert wet >= dry


def test_further_is_never_faster(trained_model):
    near = trained_model.predict(make_order(distance_km=2))[0]
    far = trained_model.predict(make_order(distance_km=12))[0]
    assert far > near


def test_error_is_better_than_guessing(model_mae, baseline_mae):
    assert model_mae < baseline_mae
'''

(WORK / "test_model.py").write_text(model_test_py, encoding="utf-8")
print("wrote", WORK / "test_model.py")
```

```
wrote work/test_model.py
```

Those tests need a trained model. Rather than train inside every test, pytest lets you prepare shared things once, in a file called `conftest.py` . Each prepared thing is called a **fixture**, and a test asks for one just by naming it as an argument.

Step 12 --- Share setup with conftest.py

`conftest.py` is a special name pytest looks for automatically. You never import it. Anything decorated with `@pytest.fixture` in it can be requested by name from any test.

In [14]:

```
conftest_py = '''"Shared setup for every test in this folder."

from pathlib import Path

import pandas as pd
import pytest
import yaml
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split

HERE = Path(__file__).resolve().parent


@pytest.fixture(scope="session")
def config():
    with open(HERE / "config.yaml", encoding="utf-8") as fh:
        return yaml.safe_load(fh)


@pytest.fixture(scope="session")
def split(config):
    data_path = HERE.parent.parent / "data" / "delivery_times.csv"
    orders = pd.read_csv(data_path)
    X = orders[config["data"]["features"]]
    y = orders[config["data"]["target"]]
    return train_test_split(
        X, y,
        test_size=config["split"]["test_size"],
        random_state=config["split"]["seed"],
    )


@pytest.fixture(scope="session")
def trained_model(split):
    X_tr, _, y_tr, _ = split
    return LinearRegression().fit(X_tr, y_tr)


@pytest.fixture(scope="session")
def model_mae(trained_model, split):
    _, X_te, _, y_te = split
    return mean_absolute_error(y_te, trained_model.predict(X_te))


@pytest.fixture(scope="session")
def baseline_mae(split):
    _, X_te, y_tr, y_te = split
    guess = [y_tr.mean()] * len(y_te)
    return mean_absolute_error(y_te, guess)
'''

(WORK / "conftest.py").write_text(conftest_py, encoding="utf-8")
run_pytest("-q")
```

```
============================= test session starts ==============================
platform linux -- Python 3.13.15, pytest-8.4.2, pluggy-1.6.0
rootdir: /content/work
plugins: anyio-4.14.2, typeguard-4.6.0, langsmith-0.12.1
collected 7 items

test_model.py ...                                                        [ 42%]
test_orders.py ....                                                      [100%]

============================== 7 passed in 0.04s ===============================

exit code: 0 (0 means every test passed)
```

Out[14]:

```
CompletedProcess(args=['/usr/bin/python3', '-m', 'pytest', '-v', '-q'], returncode=0, stdo
ut='============================= test session starts ==============================\nplat
form linux -- Python 3.13.15, pytest-8.4.2, pluggy-1.6.0\nrootdir: /content/work\nplugins:
anyio-4.14.2, typeguard-4.6.0, langsmith-0.12.1\ncollected 7 items\n\ntest_model.py ...
[ 42%]\ntest_orders.py ....                                                      [100%]\n\
n============================== 7 passed in 0.04s ===============================\n', stde
rr='')
```

Seven tests now: four about the helper functions, three about the model's behaviour. All from one command.

Your turn

The walkthrough above is finished. Now you write some code.

There are **3 tasks**. Each one is small. Each one has a hint. Do them in order.

Where you see `# TODO`, replace that line with your own code. Do not delete the variable name on the left of the `=` sign --- the self-check at the bottom looks for exactly that name.

When you have tried all three, run the **self-check** cell at the end. It prints a table telling you which tasks are correct. You can run it as many times as you like.

Task T1 --- Add a setting, and use it

Add a new section to the config called `training`, with one setting inside it: `max_rows`, set to `300`.

Then write a function `train_limited(config, data_path)` that trains on only the first `max_rows` orders of the dataset and returns the MAE.

Store the result of calling it in `T1_mae`.

> **Hint.** You already have `config` as a dictionary, so you can add to it directly: `config["training"] = {"max_rows": 300}`. In the function, read the CSV then cut it down with `orders.head(config["training"]["max_rows"])` before splitting.

In [15]:

```
# TODO: add a "training" section with max_rows set to 300
config["training"] = {"max_rows": 300}


def train_limited(config, data_path):
    orders = pd.read_csv(data_path)
    orders = orders.head(config["training"]["max_rows"])

    X = orders[config["data"]["features"]]
    y = orders[config["data"]["target"]]

    X_tr, X_te, y_tr, y_te = train_test_split(
        X,
        y,
        test_size=config["split"]["test_size"],
        random_state=config["split"]["seed"],
    )

    model = LinearRegression().fit(X_tr, y_tr)
    return mean_absolute_error(y_te, model.predict(X_te))


T1_mae = train_limited(config, DATA)
print("MAE on the first 300 orders:", T1_mae)
```

```
MAE on the first 300 orders: 2.1781751077589746
```

Task T2 --- Write a test of your own

Add a **new test file** `work/test_speed.py` containing two tests for `minutes_per_km` from `orders.py`:

1. `test_slow_delivery` --- 60 minutes over 6 km must give 10.0
2. `test_negative_distance_raises` --- a distance of `-3` must raise `ValueError`

Then run the whole suite and store the finished process in `T2_run`.

> **Hint.** Copy the shape of `test_orders.py` from Step 8. For the second test you need `with pytest.raises(ValueError):` around the call. Run everything with `T2_run = run_pytest("-q")`.

In [16]:

```
speed_test_py = '''"My own tests for minutes_per_km."

import pytest

from orders import minutes_per_km


def test_slow_delivery():
    assert minutes_per_km(60, 6) == 10.0


def test_negative_distance_raises():
    with pytest.raises(ValueError):
        minutes_per_km(60, -3)
'''

(WORK / "test_speed.py").write_text(speed_test_py, encoding="utf-8")

T2_run = run_pytest("-q")
```

```
============================= test session starts ==============================
platform linux -- Python 3.13.15, pytest-8.4.2, pluggy-1.6.0
rootdir: /content/work
plugins: anyio-4.14.2, typeguard-4.6.0, langsmith-0.12.1
collected 9 items

test_model.py ...                                                        [ 33%]
test_orders.py ....                                                      [ 77%]
test_speed.py ..                                                         [100%]

============================== 9 passed in 0.04s ===============================

exit code: 0 (0 means every test passed)
```

Task T3 --- A behaviour test for the model

Add one more test to `work/test_model.py` --- **append**, do not replace the file --- called `test_heavier_traffic_is_never_faster`.

Using the `trained_model` fixture, it must assert that an order in traffic level 3 is predicted to take at least as long as the same order in traffic level 1.

Run the suite again and store the process in `T3_run`.

> **Hint.** The file already has a `make_order(...)` helper --- use `make_order(traffic_level=1)` and `make_order(traffic_level=3)`. Open the file with `open(..., "a", encoding="utf-8")` to append. Remember the test function must take `trained_model` as its argument.

In [17]:

```
extra_test = '''

def test_heavier_traffic_is_never_faster(trained_model):
    level_1 = trained_model.predict(make_order(traffic_level=1))[0]
    level_3 = trained_model.predict(make_order(traffic_level=3))[0]

    assert level_3 >= level_1
'''

with open(WORK / "test_model.py", "a", encoding="utf-8") as fh:
    fh.write(extra_test)

T3_run = run_pytest("-q")
```

```
============================= test session starts ==============================
platform linux -- Python 3.13.15, pytest-8.4.2, pluggy-1.6.0
rootdir: /content/work
plugins: anyio-4.14.2, typeguard-4.6.0, langsmith-0.12.1
collected 10 items

test_model.py ....                                                       [ 40%]
test_orders.py ....                                                      [ 80%]
test_speed.py ..                                                         [100%]

============================== 10 passed in 0.05s ==============================

exit code: 0 (0 means every test passed)
```

Self-check

Run the cell below to mark your work.

In [18]:

```
# ---------------------------------------------------------------------
# SELF-CHECK --- run this when you have attempted the tasks above.
# It never breaks your notebook. A task you have not done yet simply
# shows FAIL.
# ---------------------------------------------------------------------

_results = []


def _check(label, fn):
    """Evaluate one graded condition without ever raising."""
    try:
        ok = bool(fn())
    except Exception:
        ok = False
    _results.append((label, ok))


_check('T1 | config has a training section with max_rows 300', lambda: config['training']['max_rows'] ==
= 300)
_check('T1 | T1_mae is a sensible error in minutes', lambda: 0 < float(T1_mae) < 15)
_check('T1 | train_limited really used only 300 rows', lambda: abs(float(train_limited({**config, 'trai
ning': {'max_rows': 300}}, DATA)) - float(T1_mae)) < 1e-9 and abs(float(train_limited({**config, 'train
ing': {'max_rows': 600}}, DATA)) - float(T1_mae)) > 1e-9)
_check('T2 | test_speed.py exists and names both tests', lambda: 'def test_slow_delivery' in (WORK / 't
est_speed.py').read_text(encoding='utf-8') and 'def test_negative_distance_raises' in (WORK / 'test_spe
ed.py').read_text(encoding='utf-8'))
_check('T2 | both tests really assert something', lambda: 'assert' in (WORK / 'test_speed.py').read_tex
t(encoding='utf-8') and 'pytest.raises' in (WORK / 'test_speed.py').read_text(encoding='utf-8'))
_check('T2 | the whole suite passes (exit code 0)', lambda: T2_run.returncode == 0)
_check('T2 | at least 9 tests were collected', lambda: any(str(n) + ' passed' in T2_run.stdout for n in
range(9, 40)))
_check('T3 | the new test asserts something and uses the fixture', lambda: 'test_heavier_traffic_is_nev
er_faster(trained_model)' in (WORK / 'test_model.py').read_text(encoding='utf-8') and (WORK / 'test_mod
el.py').read_text(encoding='utf-8').rsplit('def test_heavier', 1)[1].count('assert') >= 1)
_check('T3 | the whole suite still passes', lambda: T3_run.returncode == 0)

print("=====================================================================")
print("SELF-CHECK   Practical 05 --- Settings in a File, Bugs Caught by a Robot")
print("=====================================================================")
for _label, _ok in _results:
    print(f"  [{'PASS' if _ok else 'FAIL'}]  {_label}")
print("---------------------------------------------------------------------")
_passed = sum(1 for _, _ok in _results if _ok)
print(f"  {_passed} of {len(_results)} checks passed")
print("=====================================================================")
if _passed == len(_results):
    print("Well done. Save the notebook and submit it.")
else:
    print("Read the FAIL lines above, fix those tasks, run this cell again.")
```

```
=====================================================================
SELF-CHECK   Practical 05 --- Settings in a File, Bugs Caught by a Robot
=====================================================================
  [PASS]  T1 | config has a training section with max_rows 300
  [PASS]  T1 | T1_mae is a sensible error in minutes
  [PASS]  T1 | train_limited really used only 300 rows
  [PASS]  T2 | test_speed.py exists and names both tests
  [PASS]  T2 | both tests really assert something
  [PASS]  T2 | the whole suite passes (exit code 0)
  [PASS]  T2 | at least 9 tests were collected
  [PASS]  T3 | the new test asserts something and uses the fixture
  [PASS]  T3 | the whole suite still passes
---------------------------------------------------------------------
  9 of 9 checks passed
=====================================================================
Well done. Save the notebook and submit it.
```

What to submit

1. This notebook, with every cell run and its output visible.
2. Your `work/config.yaml` and `work/test_orders.py`.
3. In a markdown cell, one sentence explaining why `assert mae == 2.03` would be a bad test.

Name your file `PXX_<your-roll-number>.ipynb` before you upload it.

How this practical is marked

What is marked	Marks
Walkthrough run end to end, all tests passing	3
Task T1 --- a new setting added and used	2
Task T2 --- a test you wrote that passes	3
Task T3 --- a behaviour test for the model	2
Total	**10**

Read more

- pytest --- Getting started --- https://docs.pytest.org/en/stable/getting-started.html (https://docs.pytest.org/en/stable/getting-started.html)
- pytest --- Fixtures --- https://docs.pytest.org/en/stable/explanation/fixtures.html (https://docs.pytest.org/en/stable/explanation/fixtures.html)
- PyYAML documentation --- https://pyyaml.org/wiki/PyYAMLDocumentation (https://pyyaml.org/wiki/PyYAMLDocumentation)
- YAML specification --- indentation rules --- https://yaml.org/spec/1.2.2/ (https://yaml.org/spec/1.2.2/)